Prioritized NFR

1. **<u>Security</u>**

   Description

   We ensured that all passwords are stored in hash form in the server. In addition, routing to roles (patient/doctor/administrator) is handled properly using authentication cookies. Furthermore, no information is stored locally. Instead, all profile information is accessed through /profile/me routes, which verify authentication cookies

   Why?

   Security is important in most applications, especially those with login features. Moreover, our application may contain medical records and documents that are confidential and must not be publicly accessible. Therefore, security is highly important for this application, as it involves users' private information such as passwords, medical records, medical documents, and messages between patients and doctors.

2. **<u>Usability</u>**

   Description

   We spent most of our time brainstorming the application's user interfaces to ensure they are intuitive and easy to use for all users. For instance, the UI primarily includes buttons with clear text describing their functions, which improves usability and make it more responsive.

   Why?

   Our target users are deaf individuals and doctors in Africa. Therefore, we aim to make the application as easy to use as possible, minimizing additional burden on users. Doctors should be able to focus on treating patients rather than experiencing difficulties when using the application. Similarly, patients should be able to easily find doctors, book appointments, and perform other essential tasks.

3. **<u>Maintainability</u>**

   Description

   We wrote clean and modular code and applied design patterns where appropriate. In addition, every function name follows a consistent grammar convention, starting with a verb. We also ensured that the code is self-explanatory and added comments for more technical functions where necessary.

   Why?

   This project is a collaboration with Utlwa, a non-profit organization that supports deaf people in Africa. After the course ends, the project will be handed over to the founder, and we will not continue its development. Therefore, any future updates or bug fixes will be handled by other developers. As a result, we prioritize maintainability to ensure that the code is easy to read, understand, and modify for future improvements.

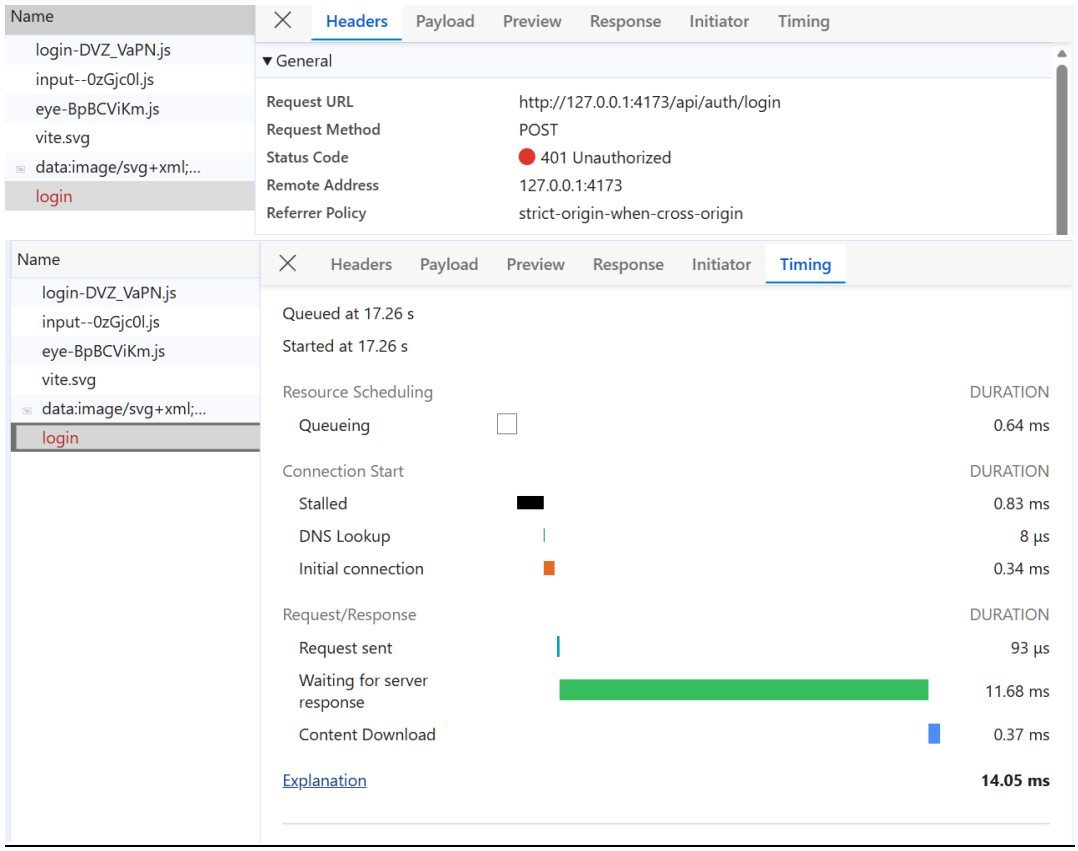
Trade-offs

1. **Security over Simplicity/Convenience**
   We decided to focus on security by utilizing authentication cookies and protected routes with authenticated `/profile/me` requests instead of locally stored authentication data. This trade-off makes our authentication system more complicated to implement and maintain.
   We are okay with this trade-off because our system will be dealing with medical and personal information. In return, we get better access control and security for our users' data.
2. **Usability and maintainability over rapid feature expansion**
   We decided to focus on making our system more usable and maintainable by spending more time designing a good interface and a well-organized structure to our code instead of spending all our time adding more features to our system. This trade-off means our system may not be able to implement all the features we wanted to implement within our time constraint.
   We are okay with this trade-off because our system should be easy to use for our patients and doctors, and it will also be passed on to future programmers. In return, we get a better interface and a more maintainable system.

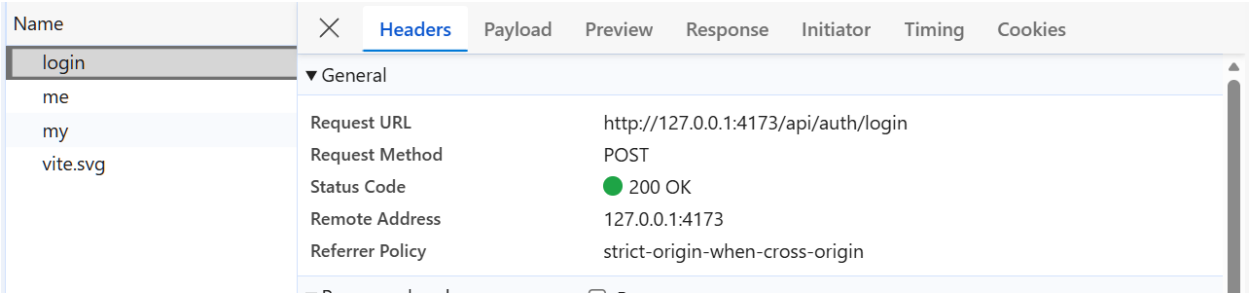
Performance Testing Report

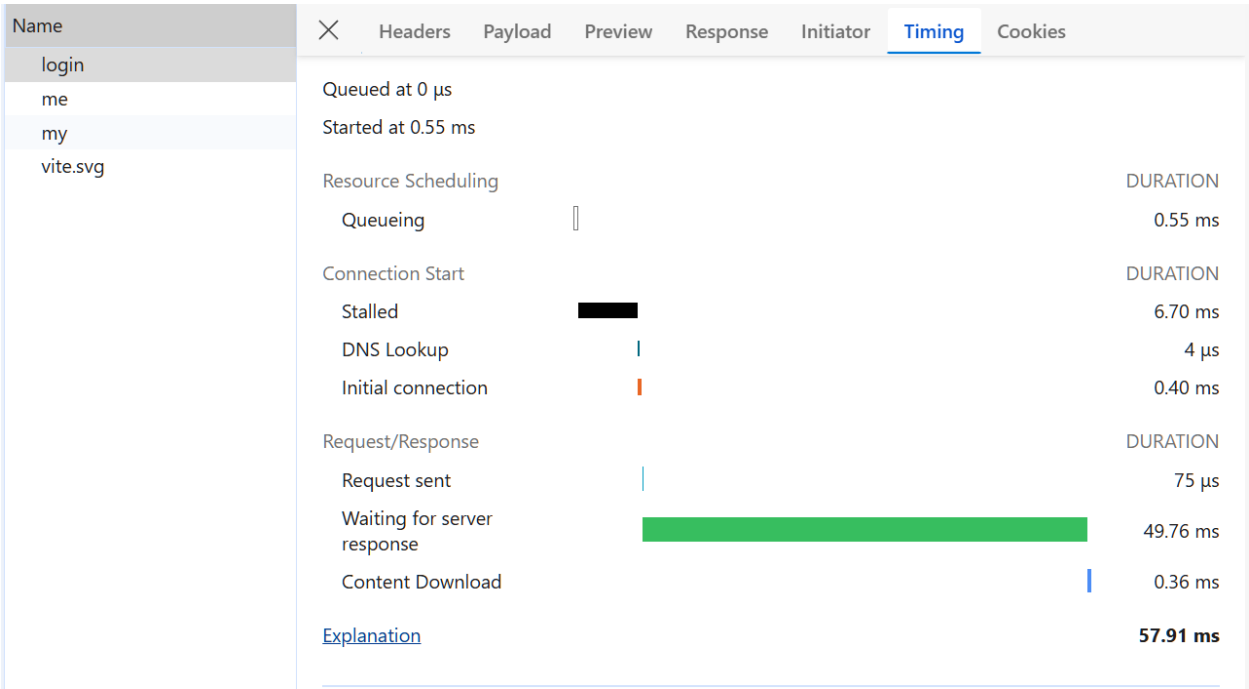
For performance testing, we tested and took screenshots for the core functionalities like Login, Doctor Recommendation List, and Booking.

- <u>Login (failed: no such email). Response time: 10ms</u>
  PASS [requirement: < 2s as stated in RPM.md]

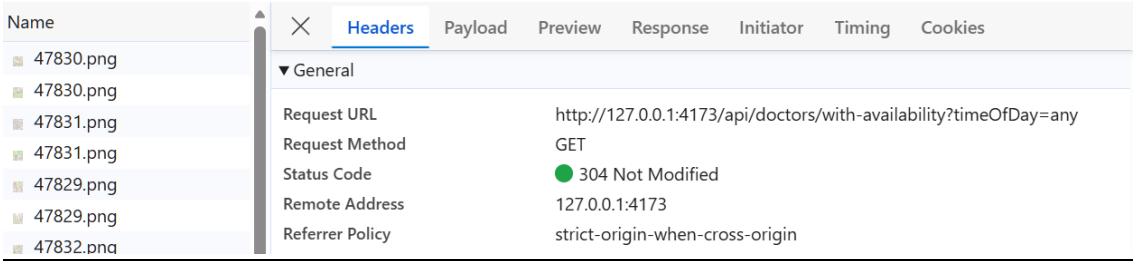


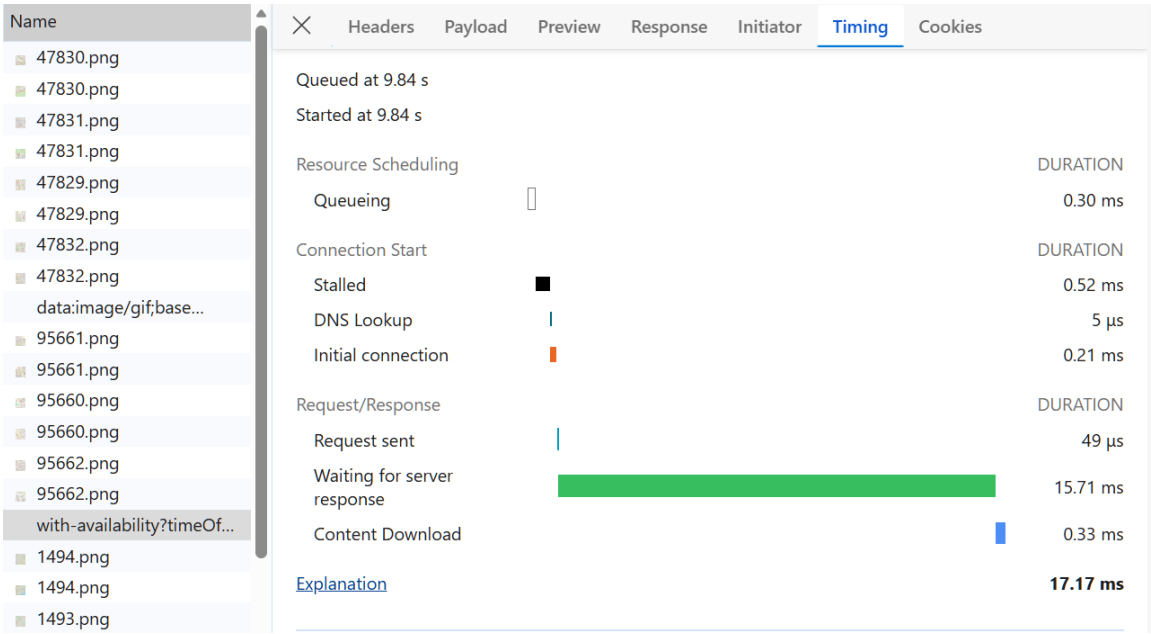
- <u>Login (success). Response time: 57.91ms</u>
  PASS [requirement: < 2s as stated in RPM.md]



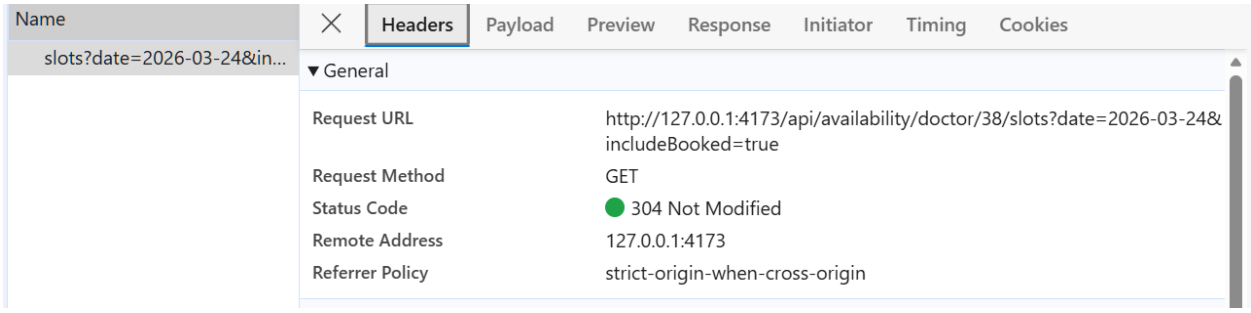


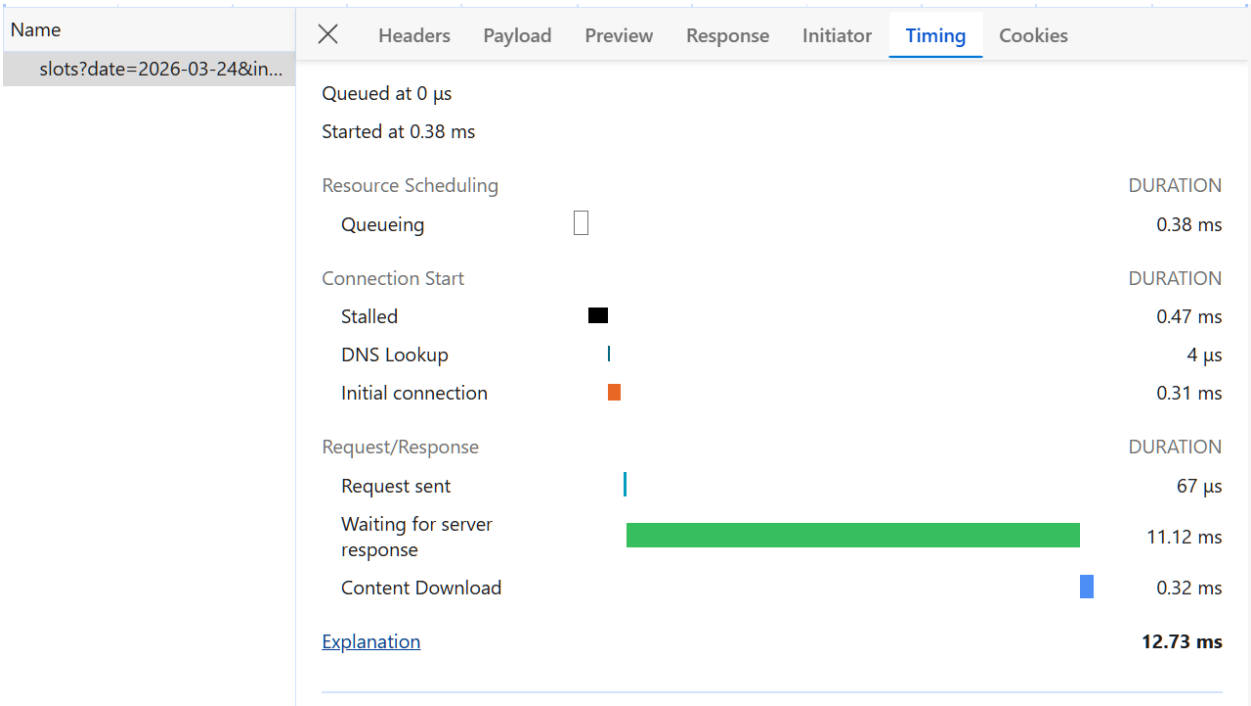
- Doctor Recommendation Display. Response time: 17.7ms
  PASS [requirement: < 3s as stated in RPM.md]



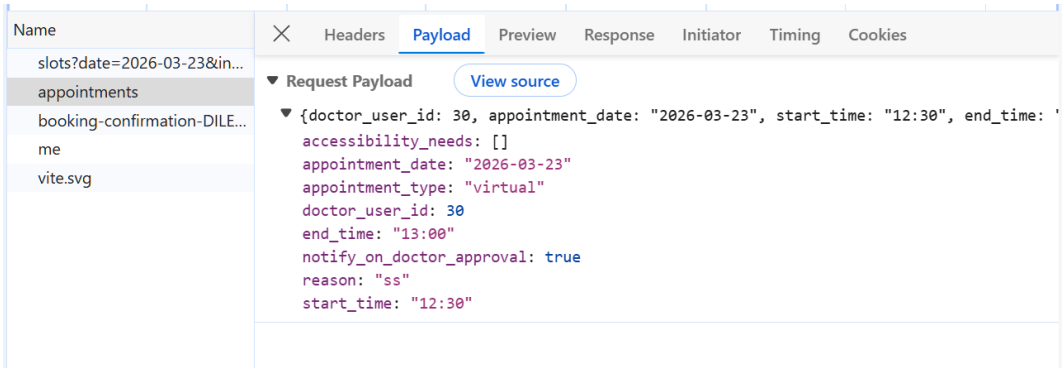


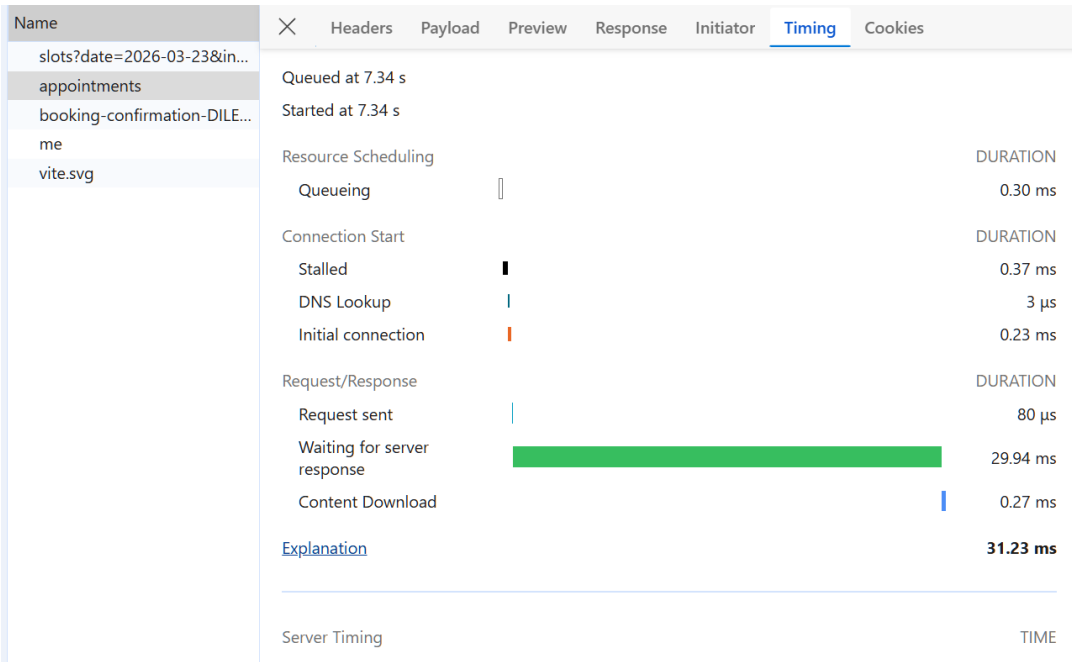
- Patient Get Doctor's Availability. Response time: 12ms
  PASS [requirement: < 2s as stated in RPM.md]



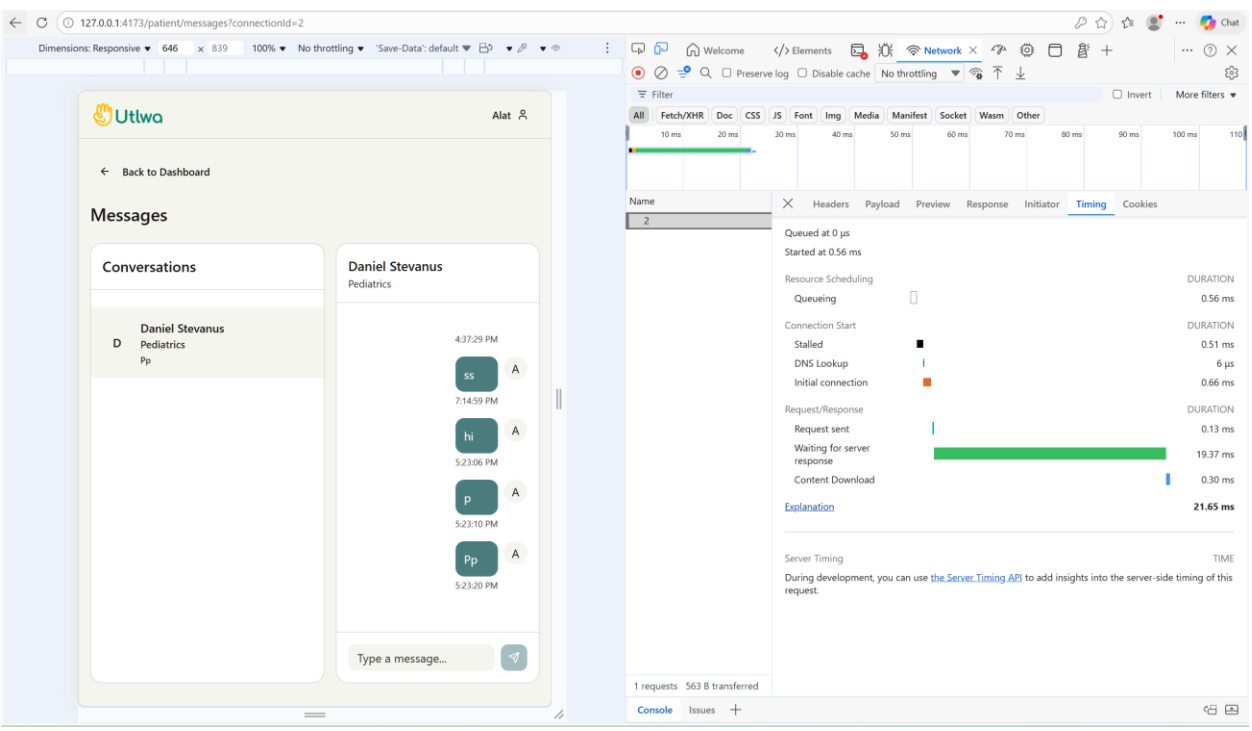


- Patient Book Appointment
  PASS [requirement: < 2s as stated in RPM.md]

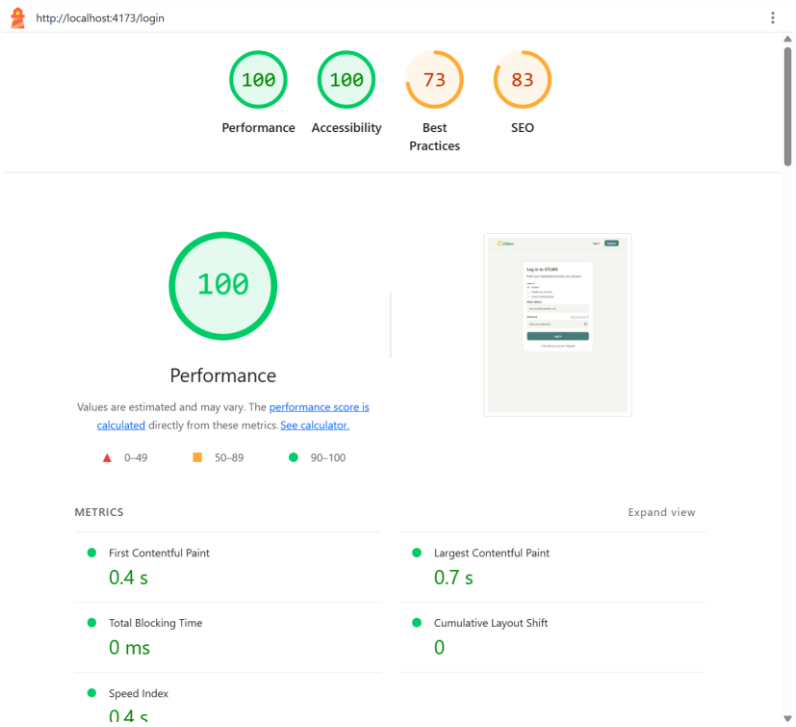




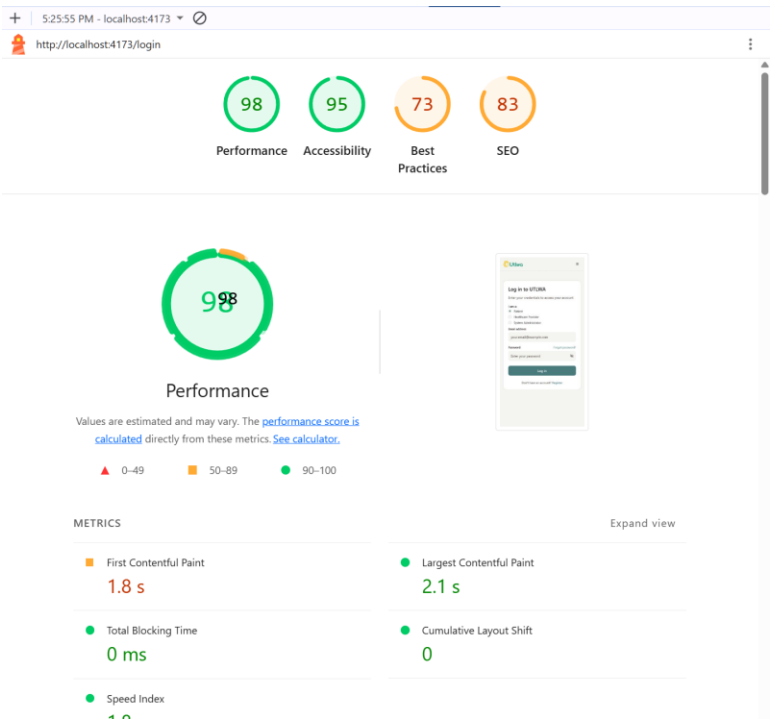
- Sending message to Doctor/Patient. Response time: 21.47ms
PASS [requirement: < 2s as stated in RPM.md]



Lighthouse Performance Test [Desktop]



Lighthouse Performance Test [Mobile]



The lighthouse performance test show the score of 98-100 which indicate good frontend performance, meaning the page loaded very efficiently with minimal blocking, stable layout behavior, and strong rendering speed. One improvement we can do is to make the Utlwa icon request from our internal server rather than from requesting it externally from utlwa.com. This will be implemented in future release.

Explanation on how system meets performance criteria

The system also meets the performance criteria that are described in the RPM documents. For instance, it is expected that most core functionalities would take no more than 2 seconds to complete, and it can be seen from the test results that the system is responsive. Additionally, we also tested the production build in Lighthouse and got 98(mobile) and 100(desktop), which indicates that the loading of the frontend is efficient. Therefore, we conclude that the system currently meets the set performance criteria and perform well with increased loads.

Security Measures and Testing

Authentication, Authorization, and Data Protection

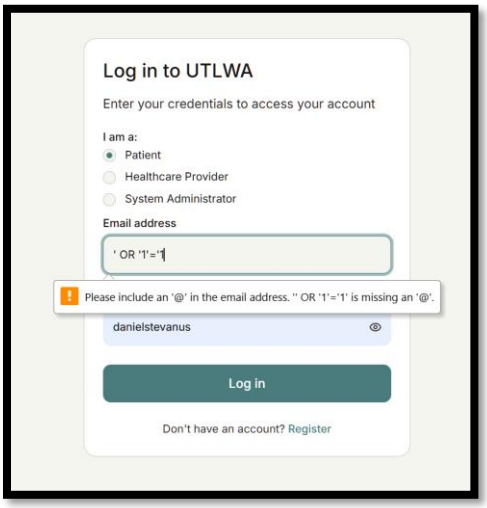
- Implemented authentication using JWT authentication cookies.
- When a user logs in or registers, the backend creates a token and stores it in an *httpOnly* cookie so it is not directly accessible through frontend JavaScript.
- Authorization through middleware that checks user role (patient, doctor) and access to their role routes.
- In addition, the frontend uses protected routes and verifies the session through /profile/me before allowing access to role-restricted pages.
- User profile information is retrieved from backend with authentication instead of being stored in user's local storage.

Security Best Practices Followed

- Passwords are hashed on the server using bcrypt before storing it to the database.
- We mostly sanity checked (check for valid inputs) in the backend flows, like registration, booking, and availability setup, to reject invalid or malformed input before processing it
- The authentication cookie is httpOnly, and it I secure in production, meaning it is intended to be sent only over HTTPS in production.

Basic Security Testing

- For SQL injection, tested input ' OR '1'='1 in login and form fields. The application did not grant unauthorized access or return unexpected data. This is supported by the backend that uses Sequelize ORM methods instead of manually handling users' SQL queries.
  Frontend check (SQL Injection)

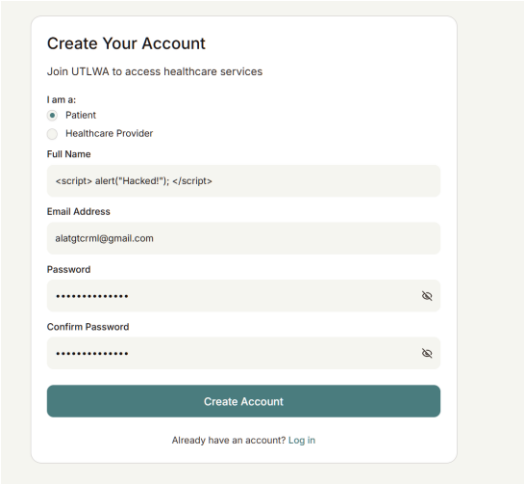


Backend check (SQL Injection)

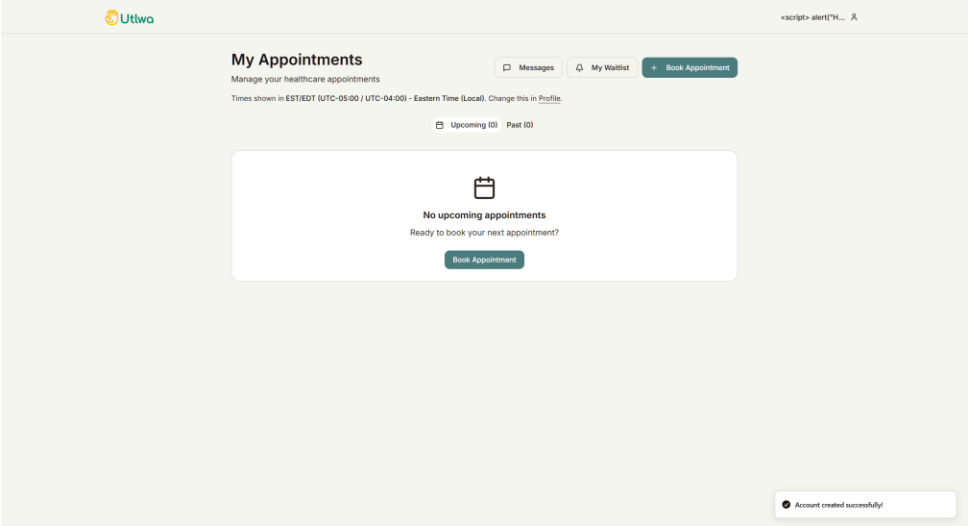
```python
DEFAULT_URL = "http://localhost:4000/api/auth/login"
DEFAULT_PAYLOAD = "' OR '1'='1"
def send_request(url: str) -> int:
    payload = {
        "email": DEFAULT_PAYLOAD,
        "password": DEFAULT_PAYLOAD,
        "role": "admin",
    }

    data = json.dumps(payload).encode("utf-8")
    request = urllib.request.Request(
        url,
        data=data,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
```

```
danielstevanus88@Dan:~/CSCC01/kindos-project$ python3 security_test_sql_injection.py
Status: 401
Response:
{"error":"Authentication failed."}
```

- For XSS, we tested input like <script>alert('xss')</script> in text fields and checked whether it executed when displayed. Frontend rendering is done through React, which check for the content and skip them by default, so user input was generally displayed as text rather than executed code.


Create Your Account
Join UTLWA to access healthcare services
I am a:
Patient
Healthcare Provider
Full Name
<script> alert("Hacked!"); </script>
Email Address
alatgtcrml@gmail.com
Password
Confirm Password
Create Account
Already have an account? Log in

-


Utlwa
<script> alert("H...
My Appointments
Manage your healthcare appointments
Messages
My Waitlist
Book Appointment
Times shown in EST/EDT (UTC-05:00 / UTC-04:00) - Eastern Time (Local). Change this in Profile.
Upcoming (0)
Past (0)
No upcoming appointments
Ready to book your next appointment?
Book Appointment
Account created successfully!

Scalability & Availability Considerations

Explanation of how the system handles increased users or data

Our project codes are designed in a modular way, that separate frontend, backend, and database layers, which makes it easy to extend and manage. The backend uses PostgreSQL with Sequelize, and the database configuration already includes a connection pool, which helps the application manage multiple database requests more efficiently.

In addition, doctor verification documents are stored using Cloudflare R2 object storage, which reduces reliance on local file storage and helps the system handle more uploaded files as the platform grows. However, the current version is still intended for small to moderate usage rather than large production traffic.

One current limitation is the real-time messaging system with Socket.io. The current implementation assumes a single backend instance, which means it cannot scale well if we have multiple servers. We identified this as a scalability limitation and are looking into possible solutions for Sprint 4.

Load Balancing or Caching

Not applicable.

Deployment Strategy and Uptime Considerations

- We will use the Split Deployment Strategy that fits UTLWA's (our collab company) existing website. The founder's main landing page can remain hosted on WordPress at utlwa.com, while our booking system is deployed separately on a subdomain like booking.utlwa.com. This approach fits the project design and architecture, since the application is built with React (frontend), Express (backend), and PostgreSQL (database), not Wordpress with plugins. It also allows the founder to keep their existing landing page website while integrating the booking platform through links/buttons. **This also aligns with the founder expectations that we have discussed on the initial meeting.**
- For uptime considerations, the backend includes a basic health-check endpoint and shutdown handling, which supports basic reliability during deployment and maintenance. However, the current project does not yet include advanced high-availability features such as redundancy or uptime monitoring, so long-term reliability would still depend on the final hosting platform and production infrastructure.